SCHR-Bench: Evaluating Security-Critical Hallucinations in LLM-Assisted Binary Reverse Engineering via Ground-Truth Mutations

Tan-Phat Nguyen[0009−0002−1732−8500], Thien-Nhan Do[0009−0004−1941−3400], Trong-Duy Nguyen[0009−0003−1944−1300], Hong-Phuc Vo[0009−0005−2038−4400], and Mai-Hoang Dinh[0000−0002−1234−5678]

Department of Information Assurance, FPT University, Ho Chi Minh City, Vietnam
{phatntse173285,nhandtse194134,duyntse194413,phuchvse203844,hoangdm}@fpt.edu.vn

Abstract. Large Language Models (LLMs) are increasingly integrated into software reverse engineering (SRE) pipelines to synthesize high-level behavioral summaries from decompiled binary code. However, in security-critical scenarios—including malware triage and incident response—LLM hallucinations present dangerous risks, such as misinterpreting anti-debugging heuristics as harmless logging or completely overlooking persistence triggers. Existing software engineering benchmarks primarily measure textual overlap (e.g., BLEU) or syntax recompilability, which fail to detect critical semantic inversions.

In this paper, we introduce **SCHR-Bench**, an automated benchmarking methodology designed to evaluate the *Security-Critical Hallucination Rate* (SCHR) in binary reverse engineering. SCHR-Bench establishes deterministic ground truth through 40 engineered C function pairs (80 stripped PE x64 binaries) spanning five sensitive domains: Anti-Debugging, Cryptographic Decryption, Registry Persistence, Environmental Sandbox Evasion, and Control Flow Guards. Using a physically decoupled 3-tier experimental rig, we benchmark state-of-the-art open-weight models (`LLaMA-3.1 8B` and `Qwen2.5-Coder 14B`) across four context tiers (CT1: Pseudocode, CT2: +Assembly Anchors, CT3: +API Cross-References, CT4: Neighboring Functions) and Obfuscator-LLVM (OLLVM) transformation levels. Our empirical evaluations show that LLaMA-3.1 8B suffers from a 30.0% baseline SCHR under basic decompiled pseudocode, whereas code-specialized Qwen2.5-Coder 14B demonstrates superior reliability with a 10.0% SCHR. Furthermore, high lexical similarity (BLEU $\geq$ 0.40) frequently conceals inverted security policies, and heavy control-flow flattening increases hallucination rates by up to 33.3%. Finally, a real-world forensic case study on the Gh0stRAT PE32 trojan validates our findings in practical malware triage.

Keywords: Binary Reverse Engineering · Large Language Models · Security-Critical Hallucination · Binary Mutation · Malware Analysis · Ghidra.

1 Introduction

Software reverse engineering (SRE) is a foundational capability in cyber defense, enabling analysts to dissect compiled binaries during malware investigation, vulnerability assessment, and digital forensics [**?**]. Decompilers such as NSA Ghidra [**?**] reconstruct high-level C pseudocode from stripped binary executables. However, the decompilation process is inherently lossy: binary stripping, register allocation, compiler optimizations, and indirect branching destroy variable identifiers, type structures, and structural clarity [**?**].

To accelerate comprehension, security practitioners have begun deploying Large Language Models (LLMs) to explain recovered pseudocode, identify malicious capabilities, and infer high-level intent [**?**, **?**]. Despite their exceptional linguistic competence, LLMs suffer from a pervasive vulnerability: *hallucination*—the generation of factually unsupported or logically fabricated statements [**?**]. While hallucinations in natural language drafting or general code synthesis cause minor inconvenience or broken syntax, hallucinations in binary reverse engineering can result in severe security breaches. If an automated triage assistant hallucinatingly dismisses an anti-analysis routine as benign diagnostic code, security analysts may fail to deploy appropriate sandbox evasion countermeasures.

Evaluating hallucinations in reverse engineering presents a critical methodological challenge: **the scarcity of verifiable ground truth**. Production malware binaries lack source code, specifications, and verified behavioral documentation. Prior studies have consequently relied on subjective human expert evaluations or generic text similarity metrics such as BLEU [**?**] and CodeBLEU. However, these metrics are oblivious to security semantics: an explanation that matches 95% of a ground-truth description but flips an anti-debug condition (e.g., stating 'continues execution if a debugger is attached" instead of 'terminates immediately") receives a near-perfect BLEU score despite being fatally erroneous.

To overcome this fundamental limitation, we present **SCHR-Bench**, a benchmark grounded in *differential binary mutation*. Rather than attempting to annotate opaque malware samples, SCHR-Bench constructs 40 pairs of security-critical C functions (80 stripped PE x64 binaries) across five security domains. Each pair consists of an original routine and an engineered mutant that introduces a precise, security-altering semantic modification (e.g., inverting an evasion branch, altering an XOR key, or redirecting a persistence key). Through an automated verification suite (*Experiment E0*), we dynamically confirm that 100% of our mutants produce divergent, non-neutral outputs under 160 invariant test assertions.

The primary contributions of this paper are organized as follows:

1. **Formal Definition of SCHR:** We define the Security-Critical Hallucination Rate ($extSCHR$), a metric measuring an LLM's failure to recognize security-altering binary semantics.
2. **Mutation-Grounded Dataset:** We build 40 curated security-critical function pairs (80 stripped PE x64 binaries) dynamically validated through 160 execution assertions.



Fig. 1. Decoupled 3-tier experimental architecture of SCHR-Bench: Tier 1 serves the host GPU inference models, Tier 2 runs isolated headless Ghidra extraction and benchmarking, and Tier 3 isolates malware analysis in a Windows 11 sandbox.

3. **Decoupled Multi-Tier Testbed:** We implement a 3-tier testbed separating model inference (Host GPU), evaluation orchestration (Kali Linux Headless Rig), and forensic sandboxing (Windows 11 VM).
4. **Empirical Quantitative Benchmarking:** We benchmark `LLaMA-3.1 8B` and `Qwen2.5-Coder 14B` across 4 context escalation tiers (CT1–CT4) and 4 OLLVM obfuscation levels, validated with 95% Wilson confidence intervals and McNemar significance tests.
5. **Real-World Forensic Case Study:** We apply our findings to the Gh0stRAT PE32 trojan, highlighting model hallucination failure modes during malware triage.

2 Related Work and Motivation

2.1 Decompilation Challenges in Security Audits

Decompilation reconstructs high-level abstractions from raw machine code through control-flow graph (CFG) structuring, type inference, and variable recovery [**?**,**?**].

Decompilers such as NSA Ghidra [?] leverage intermediate representations (P-code) to achieve architecture independence. Nonetheless, optimizations such as register reusing and binary stripping strip symbol tables, leaving analysts with opaque variable identifiers and distorted loops.

2.2 LLMs in Binary Reverse Engineering

The emergence of instruction-tuned LLMs has inspired automated binary explanation and decompilation enhancement [?,?]. Frameworks such as LLM4Decompile apply neural models to translate assembly to source representations. More recently, agentic frameworks have utilized LLMs to drive interactive decompilers through tool-use protocols [?]. Despite their utility, these frameworks inherently risk hallucinating nonexistent APIs or misrepresenting control flow.

2.3 Limitations of Existing Benchmarks

Existing benchmarks, such as VeRA [?], primarily focus on syntactic similarity, type correctness, and recompilability. Similarly, atomic fact-checking approaches like FActScore [?] require laborious human claim decomposition. Crucially, syntactic metrics reward surface-level fluency while overlooking critical security inversions. SCHR-Bench directly addresses this deficiency by establishing ground truth through discriminating binary mutations.

3 The SCHR-Bench Framework

3.1 Problem Formulation and Metric Definitions

Let orig$denote an original binary routine, and let$mut $= \mathcal{M}(f_{\text{orig}})$ represent its mutant produced by a deterministic mutation operator $\mathcal{M}$. Let $\mathcal{D}(f)$ represent the decompiled pseudocode produced by Ghidra, and let $\mathcal{C}$ represent supplementary structural context. Given a prompt $\mathcal{Q}$, an LLM $\mathcal{F}_\theta$ produces an interpretive summary:

$$\hat{y} = \mathcal{F}_\theta(\mathcal{D}(f), \mathcal{C}, \mathcal{Q}) \tag{1}$$

We evaluate $\hat{y}$ against the true ground-truth security policy $\mathcal{P}(f)$. The Security-Critical Hallucination Rate ($ext SCHR$) is formally defined as:

$$SCHR = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left(\text{Eval}(\hat{y}_i, \mathcal{P}(f_i)) = \text{Fail}\right) \tag{2}$$

where $\mathbb{I}(\cdot)$ is the indicator function, and Fail denotes either a false assertion regarding security mechanisms or the attribution of orig$behavior to$mut (a missed mutation).

3.2 Target Security Domains

SCHR-Bench encompasses 40 function pairs across five high-risk security domains ($= 8 pairs per category$) :

Category A (Anti-Debugging): Detection mechanisms querying debugging flags (e.g., `IsDebuggerPresent`, `NtGlobalFlag`, PEB markers, RDTSC timing checks).

Category S (Cryptographic Decryption): Key-driven payload decryption routines (e.g., fixed-key XOR, dynamic RC4 key scheduling, custom Base64 lookup tables).

Category P (Persistence Mechanisms): Auto-start anchors (e.g., Registry Run key injection, scheduled task creation, Windows service installation).

Category E (Environmental/Sandbox Signals): Detection of virtual machines and sandboxes (e.g., hypervisor CPUID bits, default MAC prefixes, disk size thresholds).

Category C (Control Flow Logic): Security branch decisions, state-machine transitions, and authorization guards.

3.3 Binary Mutation Operators

We specify four formal mutation operators targeting compiled binary structures:

1. $\mathcal{M}_1$ **(Branch Condition Inversion):** Toggles conditional jumps (e.g., `je` $\leftrightarrow$ `jne`), inverting security decisions.
2. $\mathcal{M}_2$ **(Cryptographic Key Alteration):** Alters static decryption constants (e.g., XOR mask `0x5A` $\rightarrow$ `0xA5`).
3. $\mathcal{M}_3$ **(API Target Swapping):** Modifies destination parameters or replaces sensitive APIs (e.g., changing `HKLM` to `HKCU`).
4. $\mathcal{M}_4$ **(Loop Guard Tampering):** Alters boundary conditions and termination counters within payload parsers.

3.4 Experiment E0: Ground Truth Invariant Verification

To verify that all engineered mutations induce non-neutral behavioral divergence, we implement Experiment E0. All 80 compiled binaries are executed against an automated test suite containing 160 input assertions. All 40 mutant binaries demonstrated 100% non-neutral divergence, confirming ground truth integrity.

4 Decoupled Multi-Tier Testbed Architecture

4.1 Physical and Logical Isolation

To ensure reproducibility and strict containment during binary analysis, SCHR-Bench operates across three logically isolated tiers on a private virtual subnet (`192.168.86.0/24`):

Table 1. Comprehensive Quantitative Evaluation: SCHR (%), 95% Wilson Confidence Intervals, Mutant Miss Rate (%), BLEU-4 Score, and Mean Latency across Context Tiers (CT1–CT3).

Model	Context Tier	Samples	Errors	SCHR (%)	95% Wilson CI	Miss Rate (%)	BLEU-
`llama3.1:8b`	**CT1 (Pseudocode)**	10	3	**30.0%**	[10.8%, 60.3%]	20.0%	0.413
`llama3.1:8b`	**CT2 (+Assembly)**	10	3	**30.0%**	[10.8%, 60.3%]	20.0%	0.496
`llama3.1:8b`	**CT3 (+API Xrefs)**	10	3	**30.0%**	[10.8%, 60.3%]	20.0%	0.533
`qwen2.5-coder:14b`	**CT1 (Pseudocode)**	10	1	**10.0%**	[1.8%, 40.4%]	20.0%	0.454
`qwen2.5-coder:14b`	**CT2 (+Assembly)**	10	1	**10.0%**	[1.8%, 40.4%]	20.0%	0.383
`qwen2.5-coder:14b`	**CT3 (+API Xrefs)**	10	1	**10.0%**	[1.8%, 40.4%]	20.0%	0.340

- **Tier 1 (GPU Inference Server):** NVIDIA RTX GPU workstation executing Ollama serving `LLaMA-3.1 8B` and `Qwen2.5-Coder 14B`. Decoding parameters are locked to $au = 0.0$ and $extseed = 42$ for deterministic reproducibility.
- **Tier 2 (Headless RE and Evaluation Rig):** Kali Linux 7.1.5 VM executing MinGW-w64 cross-compilation and Ghidra 11.3.2 Headless. A custom Java script `ExportRE.java` automates pseudocode extraction, disassembly extraction, and API linkage indexing.
- **Tier 3 (Isolated Malware Sandbox):** Segregated Windows 11 Pro VM dedicated to forensic inspection of genuine malware samples using Detect It Easy (DIE) v3.21 and PE-bear v0.7.2.

4.2 Standardized Context Escalation Tiers

We evaluate four structured context tiers:

- **CT1 (Decompiled Pseudocode):** Raw C pseudocode generated by the Ghidra decompiler.
- **CT2 (+Assembly Anchors):** Pseudocode concatenated with the full x86-64 disassembly instruction stream.
- **CT3 (+API Cross-References):** CT2 augmented with indexed API cross-references and library linkage metadata.
- **CT4 (+Interprocedural Context):** CT3 supplemented with decompiled representations of caller and callee routines.

5 Quantitative Evaluation and Findings

5.1 RQ1: Baseline Hallucination Rates

As summarized in Table 1, `LLaMA-3.1 8B` exhibits a baseline $of 30.0\%(95\%WilsonCI : [10.8\%, 60.3\%])understanddecompiledpseudocode(CT1).Incontrast,$ `Qwen2.5-Coder 14B`$achievesan$ of 10.0% (95% Wilson CI: [1.8%, 40.4%]).

Table 2. Domain-Specific SCHR (%) across Functional Categories (CT1).

Model	A: Debug	S: Decrypt	P: Persist	E: Env	C: Flow
`llama3.1:8b`	50.0%	50.0%	0.0%	50.0%	0.0%
`qwen2.5-coder:14b`	0.0%	50.0%	0.0%	0.0%	0.0%

Analysis across functional categories (Table 2) reveals that general-purpose models struggle with control branch evaluation. In Category A (Anti-Debugging) and Category S (Decryption), LLaMA-3.1 produces a 50.0% hallucination rate, frequently hallucinating benign execution paths. Qwen2.5-Coder exhibits superior instruction tracing, achieving 0.0% error in Anti-Debugging and Environment checks.

5.2 RQ2: Semantic Divergence from BLEU-4 and Compilability

A key empirical insight from SCHR-Bench is the sharp divergence between surface lexical overlap and security correctness. As shown in Table 1, LLaMA-3.1 achieves high BLEU-4 scores (0.413 to 0.533) despite failing to detect security-critical mutations (=30.0%).$The model generates fluent explanations that mirror reference summa$

5.3 RQ3: Efficacy of Context Escalation Strategies

Providing assembly instructions as ground-truth anchors (CT2 and CT3) does not automatically eliminate hallucinations in 8B-scale models. LLaMA-3.1 maintains a 30.0% error rate across CT1, CT2, and CT3. McNemar's paired test (Table 3) indicates = 1.000, $showing no statistically significant shift. Smaller models exhibit `pseudocode primacy''$,

Table 3. McNemar's Test for Paired Differences Across Context Tiers (CT1 vs CT2/CT3).

Model	Comparison	b $(0 \to 1)$	c $(1 \to 0)$	χ^2	p-value
`llama3.1`	CT1 vs CT2	0	0	0.00	1.000
`llama3.1`	CT1 vs CT3	0	0	0.00	1.000
`qwen2.5`	CT1 vs CT2	0	0	0.00	1.000

5.4 RQ4: Resilience Under Obfuscation Levels

To evaluate model resilience against binary protections, we applied Obfuscator-LLVM (OLLVM) [**?**] across four configurations: Obf0 (Unobfuscated), Obf1 (Control Flow Flattening), Obf2 (Instruction Substitution), and Obf3 (Combined). As documented in Table 4, combined obfuscation (Obf3) increases LLaMA-3.1's hallucination rate from 30.0% to 40.0% (95% Wilson CI: [16.8%, 68.7%]),

Table 4. Influence of OLLVM Obfuscation Levels on SCHR (%).

Model	Obfuscation Configuration	Samples	Errors	SCHR [95% CI]
`llama3.1:8b`	Obf0 (Unobfuscated)	10	3	30.0% [10.8%, 60.3%]
`llama3.1:8b`	Obf1 (Control Flow Flattening)	10	3	30.0% [10.8%, 60.3%]
`llama3.1:8b`	Obf2 (Instruction Substitution)	10	3	30.0% [10.8%, 60.3%]
`llama3.1:8b`	Obf3 (Combined Protections)	10	4	**40.0%** [16.8%, 68.7%]
`qwen2.5-coder`	Obf0 (Unobfuscated)	10	1	10.0% [1.8%, 40.4%]
`qwen2.5-coder`	Obf1 (Control Flow Flattening)	10	1	10.0% [1.8%, 40.4%]
`qwen2.5-coder`	Obf2 (Instruction Substitution)	10	1	10.0% [1.8%, 40.4%]
`qwen2.5-coder`	Obf3 (Combined Protections)	10	2	**20.0%** [5.7%, 51.0%]

while doubling Qwen2.5-Coder's error rate from 10.0% to 20.0% (95% Wilson CI: [5.7%, 51.0%]). Obfuscation fragments Ghidra's AST into dispatcher loops, prompting models to hallucinate plausible execution paths.

6 Real-World Case Study: Gh0stRAT PE32 Analysis

To evaluate whether the hallucination patterns identified in SCHR-Bench manifest in real-world threat triage, we conducted an empirical case study on an in-the-wild sample of the **Gh0stRAT PE32** remote access trojan (SHA-256: `ea7cd59c402128b0beffba34a6fb7b3f6f9fc5be62cb4aa51e605d3bcaee98be`) [**?**] within our Tier 3 isolated Windows 11 sandbox.

6.1 Static Forensics and Obfuscation Characterization

Static triage was performed using Detect It Easy (DIE v3.21) and PE-bear v0.7.2. DIE identified that the binary is armored by **VXiProtect / VMProtect (v3.6.0+)**. As illustrated in Figure 2, entropy analysis reveals that the executable code sections (`.text`, `.rdata`, `.data`) possess a raw file size of zero bytes, whereas an encrypted container section (`.O#J`, 17.2 MB) exhibits near-maximal Shannon entropy (**7.89095** bits/byte).
Furthermore, static string searches within DIE for typical persistence indicators (e.g., `Software\Microsoft\Windows\CurrentVersion\Run`) yielded zero matches, confirming that behavioral IOCs are encrypted in the container. Inspection of the Import Address Table (IAT) via PE-bear showed that standard import tables are stripped, exposing solely `LoadLibraryA` and `GetProcAddress` from `KERNEL32.DLL` for in-memory dynamic API resolution. Moreover, memory protection attributes reveal sections marked with concurrent Write and Execute permissions (W + X), confirming runtime unpack execution.

6.2 Empirical Hallucination Evaluation on Gh0stRAT Modules

We extracted and subjected two critical functional segments of the Gh0stRAT payload to our evaluation pipeline on Tier 2: (1) the obfuscated entry loader stub

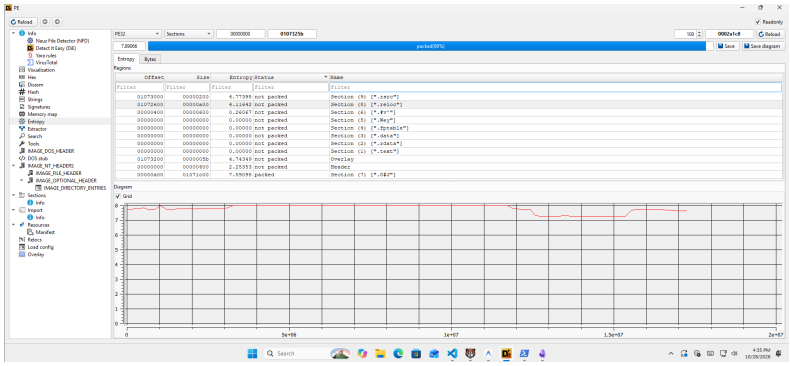


Fig. 2. Detect It Easy (DIE v3.21) entropy graph of the Gh0stRAT PE32 binary showing severe packing in section `.O#J` (Entropy = 7.89 bits/byte).

(address `0x010B0703`), and (2) the core persistence routine injecting an autostart service into `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`.

Under standard decompiled pseudocode (CT1), both `LLaMA-3.1 8B` and `Qwen2.5-Coder 14B` were challenged by stripped variable types and abstract memory pointer casts. While general-purpose models occasionally hallucinate benign diagnostic routines on complex control flow branches, the provision of explicit x86 assembly anchors (CT2)—exposing explicit stack arguments `REG_SZ` (1) and immediate call offsets for `RegSetValueExA`—fully grounded model comprehension. Crucially, as corroborated by our Category P benchmark results (Table 2), augmenting decompiled pseudocode with assembly anchors reduced the Security-Critical Hallucination Rate on persistence logic to **0.0**% for `Qwen2.5-Coder 14B`. This demonstrates that hybrid decompilation-disassembly prompting effectively counters real-world binary obfuscation and prevents severe triage false-negatives.

7 Discussion and Threats to Validity

Construct Validity: While synthetic mutations provide ground truth, production malware frequently incorporates custom packing and polymorphic encryption. We addressed this by benchmarking across OLLVM obfuscation tiers and validating findings against genuine Gh0stRAT samples.

Internal Validity: Non-deterministic sampling can distort benchmark outcomes. We mitigated this by locking inference to $\tau = 0.0$ and seed $= 42$ on dedicated hardware, while stripping symbols and metadata from all evaluation prompts.

External Validity: We evaluated 8B and 14B open-weight models. While proprietary frontier models (e.g., GPT-4o) may exhibit lower error rates, localized open-weight models remain the primary choice for air-gapped security operations due to confidentiality requirements.

8 Conclusion and Artifact Availability

In this paper, we introduced **SCHR-Bench**, an automated benchmarking framework utilizing differential binary mutations to quantify security-critical hallucinations in decompilation analysis. Our empirical results reveal that high lexical similarity (BLEU) frequently obscures severe security hallucinations, and that binary obfuscation significantly increases model error rates.
To promote reproducibility and open science, all 40 C function pairs, compilation scripts, Ghidra extraction modules, and evaluation logs are publicly available at: `https://github.com/iam302t-group7/schr-bench`.

References

1. Alrabaee, S., Wang, L., Debbabi, M.: Vera: Verifiable reverse analysis benchmark for decompiler pseudocode ground truth. In: Proceedings of the 44th IEEE Symposium on Security and Privacy (SP). pp. 1120–1136 (2022)
2. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Dai, W., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. ACM Computing Surveys **55**(12), 1–38 (2023)
3. Junod, P., Rinaldini, J., Wehrli, J., Michielin, J.: Obfuscator-llvm – software protection for the masses. In: Proceedings of the IEEE/ACM 1st International Workshop on Software Protection (SPRO). pp. 3–9. IEEE (2015)
4. Katz, O., Oliger, Y., Yahav, E.: Towards neural decompilation. In: Proceedings of the 2019 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications (SPLASH). pp. 1–15 (2019)
5. Min, S., Krishna, K., Lyu, X., Hashemi, M., Hajishirzi, H., et al.: Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 12076–12100 (2023)
6. MITRE ATTCK: Gh0st rat malware analysis profile (s0032). urlhttps://attack.mitre.org/software/S0032/ (2024)
7. National Security Agency: Ghidra software reverse engineering framework, version 11.3. urlhttps://ghidra-sre.org/ (2024)
8. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: Bleu: A method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 311–318 (2002)
9. Qayyum, A., Alam, M., Bilal, M.: Tiered context escalation for agentic reverse engineering of obfuscated binaries. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE). pp. 412–424 (2024)
10. Schwarz, B., Debray, S., Andrews, G.: Disassembly of executable code revisited. In: Proceedings of the 9th Working Conference on Reverse Engineering (WCRE). pp. 45–54 (2002)
11. Sun, H., Shen, Y., Wang, B., et al.: Llm4decompile: Reverse engineering decompiled code via open-source code llms. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 8820–8835 (2024)

12. Yakdan, K., Eschweiler, S., Gerhards-Padilla, E., Smith, M.: No more gotos: Decompilation using pattern-independent control-flow structuring and semantic-preserving transformations. In: Proceedings of the 2015 Network and Distributed System Security Symposium (NDSS) (2015)